

C05-AT3-CODING CHALLENGE

1. Employee Performance Analytics System

Concepts used

- Class and objects
- Private/public access specifiers
- Member functions
- Static data member
- Array of objects
- Dynamic array for variable department sizes
- Functions
- Finding maximum
- Average calculation
- Overall best performer
- Edge case: no employees
- Equal performance scores

```
#include <iostream>
```

```
#include <string>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
class Employee
```

```
{
```

```
private:
```

```
    int id;
```

```
    string name;
```

```
    float score;
```

```
public:
```

```
    static int totalEmployees;
```

```
Employee()
```

```
{
```

```
    id = 0;
```

```
    name = "";
```

```
    score = 0;
```

```
}
```

```
void input()
```

```
{
```

```
    cout << "Enter Employee ID: ";
```

```
    cin >> id;
```

```
    cout << "Enter Employee Name: ";
```

```
    cin >> name;
```

```
    cout << "Enter Performance Score: ";
```

```
    cin >> score;
```

```
    totalEmployees++;
```

```
}
```

```
float getScore()
```

```
{
```

```
    return score;
```

```
}
```

```
string getName()
```

```
{
```

```
    return name;
```

```
}
```

```
int getID()
```

```
{
```

```
    return id;
```

```
}
```

```

void display()
{
    cout << "ID: " << id
        << " Name: " << name
        << " Score: " << score << endl;
}

};

int Employee::totalEmployees = 0;

// Find top performer
int findTopPerformer(Employee emp[], int n)
{
    if (n == 0)
        return -1;

    int index = 0;
    for (int i = 1; i < n; i++)
    {
        if (emp[i].getScore() > emp[index].getScore())
        {
            index = i;
        }
    }

    return index;
}

// Calculate average score
float calculateAverage(Employee emp[], int n)
{
    if (n == 0)

```

```

        return 0;

float sum = 0;

for (int i = 0; i < n; i++)
{
    sum += emp[i].getScore();
}

return sum / n;
}

int main()
{
    int departments;

    cout << "Enter number of departments: ";
    cin >> departments;

    if (departments <= 0)
    {
        cout << "No departments available." << endl;

        return 0;
    }

    Employee **dept = new Employee*[departments];

    int *size = new int[departments];

    int overallDept = -1;

    int overallEmployee = -1;

    float overallBestScore = -1;

    for (int d = 0; d < departments; d++)
    {
        cout << "\n===== Department "

        << d + 1 << " =====\n";
    }
}

```

```

cout << "Enter number of employees: ";

cin >> size[d];

if (size[d] <= 0)
{
    dept[d] = nullptr;

    cout << "No employees in this department.\n";

    continue;
}

dept[d] = new Employee[size[d]];

for (int i = 0; i < size[d]; i++)
{
    cout << "\nEmployee " << i + 1 << endl;

    dept[d][i].input();
}

int top = findTopPerformer(dept[d], size[d]);

cout << "\nTop Performer:\n";

dept[d][top].display();

cout << fixed << setprecision(2);

cout << "Average Performance Score: "

    << calculateAverage(dept[d], size[d]) << endl;

// Overall best performer

if (dept[d][top].getScore() > overallBestScore)
{
    overallBestScore = dept[d][top].getScore();

    overallDept = d;

    overallEmployee = top;
}

```

```

    }

    cout << "\n===== \n";

    cout << "    OVERALL BEST EMPLOYEE\n";

    cout << "===== \n";

    if (overallDept != -1)
    {
        dept[overallDept][overallEmployee].display();

        cout << "Department: "

            << overallDept + 1 << endl;
    }

    else

    {
        cout << "No employees available.\n";
    }

    cout << "\nTotal Employees Entered: "

        << Employee::totalEmployees << endl;

    // Free memory

    for (int d = 0; d < departments; d++)
    {
        delete[] dept[d];
    }

    delete[] dept;

    delete[] size;

    return 0;
}

```

Sample Input

Enter number of departments: 2

===== Department 1 =====

Enter number of employees: 3

Employee 1

ID: 101

Name: Arun

Score: 78

Employee 2

ID: 102

Name: Ravi

Score: 92

Employee 3

ID: 103

Name: Kumar

Score: 85

===== Department 2 =====

Enter number of employees: 2

Employee 1

ID: 201

Name: Priya

Score: 88

Employee 2

ID: 202

Name: Anu

Score: 95

Sample Output

Top Performer:

ID: 102 Name: Ravi Score: 92

Average Performance Score: 85.00

Top Performer:

ID: 202 Name: Anu Score: 95

Average Performance Score: 91.50

=====

OVERALL BEST EMPLOYEE

=====

ID: 202 Name: Anu Score: 95

Department: 2

Total Employees Entered: 5

main.cpp	Run	Output
<pre>6 class Employee 7 { 8 private: 9 int id; 10 string name; 11 float score; 12 13 public: 14 static int totalEmployees; 15 16 Employee() 17 { 18 id = 0; 19 name = ""; 20 score = 0; 21 } 22 23 void input() 24 { 25 cout << "Enter Employee ID: "; 26 cin >> id; 27</pre>		<pre>Enter Employee ID: 141 Enter Employee Name: ragul Enter Performance Score: 9/10 Employee 2 Enter Employee ID: Enter Employee Name: Enter Performance Score: Employee 3 Enter Employee ID: Enter Employee Name: Enter Performance Score: Employee 4 Enter Employee ID: Enter Employee Name: Enter Performance Score: Top Performer: ID: 141 Name: ragul Score: 9 Average Performance Score: 2.25 ===== Department 2 ===== Enter number of employees: No employees in this department. ===== OVERALL BEST EMPLOYEE ===== ID: 141 Name: ragul Score: 9.00 Department: 1</pre>

2. Fleet Management System

Concepts used

- Class and objects
- Private/public members
- Member functions
- Static member
- Array of objects
- Dynamic arrays
- Functions
- Maximum distance
- Average distance
- Overall most efficient vehicle

```
#include <string>

#include <iomanip>

using namespace std;

class Vehicle

{

private:

    int vehicleID;

    string driverName;

    float distance;

public:

    static int totalVehicles;

    Vehicle()

    {

        vehicleID = 0;

        driverName = "";

        distance = 0;

    }

    void input()
```

```
{  
  
    cout << "Enter Vehicle ID: ";  
  
    cin >> vehicleID;  
  
    cout << "Enter Driver Name: ";  
  
    cin >> driverName;  
  
    cout << "Enter Distance Covered: ";  
  
    cin >> distance;  
  
    totalVehicles++;  
  
}  
  
float getDistance()  
{  
  
    return distance;  
  
}  
  
string getDriver()  
{  
  
    return driverName;  
  
}  
  
int getID()  
{  
  
    return vehicleID;  
  
}  
  
void display()  
{  
  
    cout << "Vehicle ID: " << vehicleID  
  
        << " Driver: " << driverName  
  
        << " Distance: " << distance << " km"  
  
        << endl;  
  
}
```

```

};

int Vehicle::totalVehicles = 0;

// Find vehicle with highest distance

int findHighestDistance(Vehicle vehicle[], int n)
{
    if (n == 0)
        return -1;

    int index = 0;

    for (int i = 1; i < n; i++)
    {
        if (vehicle[i].getDistance() >
            vehicle[index].getDistance())
        {
            index = i;
        }
    }

    return index;
}

// Calculate average distance

float calculateAverageDistance(Vehicle vehicle[], int n)
{
    if (n == 0)
        return 0;

    float total = 0;

    for (int i = 0; i < n; i++)
    {
        total += vehicle[i].getDistance();
    }
}

```

```

        return total / n;
    }

int main()
{
    int routes;

    cout << "Enter number of routes: ";

    cin >> routes;

    if (routes <= 0)
    {
        cout << "No routes available." << endl;

        return 0;
    }

    Vehicle **fleet = new Vehicle*[routes];

    int *size = new int[routes];

    int overallRoute = -1;

    int overallVehicle = -1;

    float bestDistance = -1;

    for (int r = 0; r < routes; r++)
    {
        cout << "\n===== Route "

            << r + 1 << " =====\n";

        cout << "Enter number of vehicles: ";

        cin >> size[r];

        if (size[r] <= 0)
        {
            fleet[r] = nullptr;

            cout << "No vehicles on this route.\n";

```

```

        continue;
    }

    fleet[r] = new Vehicle[size[r]];

    for (int i = 0; i < size[r]; i++)
    {
        cout << "\nVehicle " << i + 1 << endl;

        fleet[r][i].input();
    }

    int highest =

        findHighestDistance(fleet[r], size[r]);

    cout << "\nVehicle with Highest Distance:\n";

    fleet[r][highest].display();

    cout << fixed << setprecision(2);

    cout << "Average Distance: "

        << calculateAverageDistance(

            fleet[r], size[r])

        << " km\n";

    // Overall most efficient vehicle

    if (fleet[r][highest].getDistance() >

        bestDistance)

    {
        bestDistance =

            fleet[r][highest].getDistance();

        overallRoute = r;

        overallVehicle = highest;
    }
}

```

```

cout << "\n===== \n";

cout << "    OVERALL MOST EFFICIENT\n";

cout << "===== \n";

if (overallRoute != -1)
{
    fleet[overallRoute][overallVehicle].display();

    cout << "Route: "

        << overallRoute + 1 << endl;
}

else

{
    cout << "No vehicles available.\n";
}

cout << "\nTotal Vehicles: "

    << Vehicle::totalVehicles << endl;

// Free memory

for (int r = 0; r < routes; r++)
{
    delete[] fleet[r];
}

delete[] fleet;

delete[] size;

return 0;
}

```

Sample Input

Enter number of routes: 2

===== Route 1 =====

Enter number of vehicles: 3

Vehicle 1

Vehicle ID: 101

Driver: Arun

Distance: 450

Vehicle 2

Vehicle ID: 102

Driver: Ravi

Distance: 620

Vehicle 3

Vehicle ID: 103

Driver: Kumar

Distance: 500

===== Route 2 =====

Enter number of vehicles: 2

Vehicle 1

Vehicle ID: 201

Driver: Priya

Distance: 700

Vehicle 2

Vehicle ID: 202

Driver: Anu

Distance: 550

Sample Output

Vehicle with Highest Distance:

Vehicle ID: 102 Driver: Ravi Distance: 620 km

Average Distance: 523.33 km

Vehicle with Highest Distance:

Vehicle ID: 201 Driver: Priya Distance: 700 km

Average Distance: 625.00 km

=====

OVERALL MOST EFFICIENT

=====

Vehicle ID: 201 Driver: Priya Distance: 700 km

Route: 2

Total Vehicles: 5

main.cpp	Run	Output
<pre>1 #include <iostream> 2 #include <string> 3 #include <iomanip> 4 using namespace std; 5 6 class Vehicle 7 { 8 private: 9 int vehicleID; 10 string driverName; 11 float distance; 12 13 public: 14 static int totalVehicles; 15 16 Vehicle() 17 { 18 vehicleID = 0; 19 driverName = ""; 20 distance = 0; 21 } 22</pre>		<pre>Enter number of routes: 35 ===== Route 1 ===== Enter number of vehicles: 2 Vehicle 1 Enter Vehicle ID: 414 Enter Driver Name: ragul Enter Distance Covered: 200k Vehicle 2 Enter Vehicle ID: Enter Driver Name: Enter Distance Covered: Vehicle with Highest Distance: Vehicle ID: 414 Driver: ragul Distance: 200 km Average Distance: 100.00 km ===== Route 2 ===== Enter number of vehicles: No vehicles on this route. ===== Route 3 ===== Enter number of vehicles: No vehicles on this route.</pre>